

DESIGN AND IMPLEMENTATION OF ORDERS AND ORDER_DETAILS TABLES

1. Introduction

The Orders and Order_Details tables are designed to store information about customer orders and the products included in each order.

The **Orders table** stores the overall information about an order, such as Customer ID, Order Date, Total Amount and Order Status.

The **Order_Details table** stores the individual products included in each order along with the quantity purchased.

2. Orders Entity

The Orders entity stores the overall order information.

Attribute	Description
Order_ID	Unique order identifier
Customer_ID	Customer reference
Order_Date	Date of order placement
Total_Amount	Total bill amount
Order_Status	Pending / Shipped / Delivered

3. Order_Details Entity

The Order_Details entity stores the individual products included in an order.

Attribute	Description
Order_Detail_ID	Unique detail identifier
Order_ID	Order reference
Product_ID	Product reference
Quantity	Number of products purchased

4. Creating the ORDERS Table

The ORDERS table is created to store the overall information of customer orders.

```
SQL> CREATE TABLE ORDERS(Order_ID VARCHAR2(7),Customer_ID VARCHAR2(7),Order_Date DATE,Total_Amount NUMBER(10,2),Order_Status VARCHAR2(20));
Table created.
```

5. Establishing Primary Keys

A Primary Key is used to uniquely identify each record in a table.

The Order_ID is the Primary Key of the ORDERS table.

The Order_Detail_ID is the Primary Key of the ORDER_DETAILS table.

```
ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY(Order_ID);
```

```
ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_ORDER_DETAILS PRIMARY KEY(Order_Detail_ID);
```

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY(Order_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_ORDER_DETAILS PRIMARY KEY(Order_Detail_ID);
Table altered.
```

6. Establishing Relationships

The relationship between Customer and Orders is:

Customer (1) → Orders (Many)

One customer can place multiple orders, while each order belongs to one customer.

The relationship between Orders and Products is:

Orders (Many) → Products (Many)

This many-to-many relationship is implemented using the **ORDER_DETAILS** table.

7. Applying Foreign Key Constraint

The Order_ID in the ORDER_DETAILS table is connected to the Order_ID in the ORDERS table using a Foreign Key.

```
ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_ORDER FOREIGN
KEY(Order_ID) REFERENCES ORDERS(Order_ID);
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_ORDER FOREIGN KEY(Order_ID) REFERENCES ORDERS(Order_ID);
Table altered.
```

8. Applying NOT NULL Constraint

The Order_ID and Product_ID in the ORDER_DETAILS table are made mandatory using the NOT NULL constraint.

```
ALTER TABLE ORDER_DETAILS MODIFY(Order_ID NOT NULL,Product_ID NOT NULL);
```

```
SQL> ALTER TABLE ORDER_DETAILS MODIFY(Order_ID NOT NULL,Product_ID NOT NULL);  
Table altered.
```

9. Applying CHECK Constraints

A CHECK constraint is used to ensure that only valid values are entered into the table.

Total Amount Constraint

```
ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_AMOUNT CHECK(Total_Amount>=0);
```

This ensures that the total amount cannot be negative.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_AMOUNT CHECK(Total_Amount>=0);  
Table altered.
```

Order Status Constraint

```
ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(Order_Status  
IN('Pending','Shipped','Delivered'));
```

This allows only **Pending**, **Shipped** or **Delivered** as the order status.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(Order_Status IN('Pending','Shipped','Delivered'));  
Table altered.
```

10. Inserting Values into ORDERS

The following records are inserted into the ORDERS table.

```
SQL> INSERT INTO ORDERS VALUES('ORD0001','CUS0001',TO_DATE('24-09-2026','DD-MM-YYYY'),25000,'Pending');  
1 row created.  
  
SQL> INSERT INTO ORDERS VALUES('ORD0002','CUS0002',TO_DATE('24-09-2026','DD-MM-YYYY'),15000,'Shipped');  
1 row created.
```

11. Inserting Values into ORDER_DETAILS

The following records are inserted into the ORDER_DETAILS table.

```
SQL> INSERT INTO ORDER_DETAILS VALUES('OD00001','ORD0001','P101',2);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD00002','ORD0001','P111',1);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD00003','ORD0002','P102',2);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD00004','ORD0002','P001',1);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD00005','ORD0002','P002',2);  
1 row created.
```

12. Customer Order History

This report displays the customer name, order ID, order date, total amount and order status for each customer order.

```
SQL> SELECT CUSTOMER_NAME, ORDER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS AS STATUS FROM ORDERS;
```

CUSTOMER_NAME	ORDER_I	ORDER_DATE	TOTAL_AMOUNT	STATUS
YAMINI	ORD0001	24-SEP-26	25000	Pending
NANDY	ORD0002	24-SEP-26	15000	Shipped

13. Product-Wise Order Report

This query finds the number of times each product was ordered and the total quantity sold.

The query uses GROUP BY to group the order details based on each product.

It calculates:

- **No_Of_Times_Ordered** – Number of times the product was ordered.
- **Total_Quantity_Sold** – Total quantity of the product sold.

```
SQL> SELECT PRODUCT_ID, COUNT(*) AS NO_OF_TIMES_ORDERED, SUM(QUANTITY) AS TOTAL_QUANTITY_SOLD FROM ORDER_DETAILS GROUP BY PRODUCT_ID;
```

PRODUCT	NO_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
P001	1	1
P102	1	2
P101	1	2
P111	1	1
P002	1	2

14. Customer Purchase Analysis

This report analyses customer purchases by finding customers with maximum orders, total spending by each customer, and average order value.

```
SQL> SELECT CUSTOMER_NAME, COUNT(*) AS TOTAL_ORDERS, SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING, AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM ORDERS GROUP BY CUSTOMER_NAME;
```

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
YAMINI	1	25000	25000
NANDY	1	15000	15000

```
SQL> SELECT CUSTOMER_NAME, COUNT(*) AS TOTAL_ORDERS FROM ORDERS GROUP BY CUSTOMER_NAME HAVING COUNT(*) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_NAME));
```

CUSTOMER_NAME	TOTAL_ORDERS
YAMINI	1
NANDY	1

15. Result

The SQL queries also generate useful reports such as:

- Customer order history report
 - Product-wise order report
 - Customer purchase analysis
-

16. Conclusion

The Orders and Order_Details tables were successfully designed and implemented using SQL.

The SQL queries provide useful information about customer order history, products ordered, quantities sold and customer purchase analysis